Actividad individual 1 -

MÓDULO PROFESIONAL: Desarrollo de Interfaces (Código: 0488)

CICLO FORMATIVO: Grado Superior en Desarrollo de Aplicaciones Multiplataforma (DAM)
TIPO DE ACTIVIDAD: Proyecto Individual de Evaluación Práctica
MODALIDAD: Entregable Final

DOCUMENTO DE ESPECIFICACIONES DE PROYECTO

Título del Proyecto: Diseño, Desarrollo y Despliegue de Sistema de Gestión Multiplataforma (PWA/Híbrida).
Descripción General:
El alumno deberá diseñar y programar desde cero una aplicación de gestión orientada a un entorno empresarial, utilizando el framework **Ionic** apoyado en **Angular** o **React** (a elección del estudiante). El proyecto requiere la implementación de una arquitectura robusta, consumo asíncrono de orígenes de datos (SQL y NoSQL), control de acceso por roles, y un riguroso cumplimiento de las normativas de accesibilidad (WCAG). La aplicación debe culminar en un despliegue automatizado en un entorno de producción real.

FASES DEL PROYECTO Y PASOS A REALIZAR

La ejecución del proyecto debe seguir estrictamente las siguientes fases, evaluables según el currículo oficial de la Comunidad de Madrid.

FASE 1: Estructuración y Renderizado Declarativo (RA 1 y RA 2)

El alumno configurará la base arquitectónica (MVC/MVVM) y construirá las interfaces principales mediante programación declarativa.

- **Paso 1.1 - Sistema de Enrutamiento y Navegación:** Implementar un sistema de navegación global (Side Menu o Tabs) haciendo uso de carga diferida (*Lazy Loading*). El alumno debe configurar la protección de rutas (*Guards* o *Private Routes*) basándose en un sistema de autenticación con dos roles definidos: Administrador y Usuario Estándar.
- **Paso 1.2 - Formularios y Validaciones Asíncronas:** Construir interfaces de entrada de datos (ej. Login y Alta de registros) utilizando marcado declarativo (JSX o plantillas HTML). Se exige la aplicación de *Data Binding* (unidireccional y bidireccional) y el desarrollo de validaciones asíncronas contra un servicio (ej. verificar si un correo ya existe en base de datos) gestionando correctamente el estado del componente.

FASE 2: Componentización Visual y Lógica de Negocio (RA 3)

El alumno demostrará dominio en la reutilización de código y la separación de responsabilidades en la capa de presentación.

- **Paso 2.1 - Desarrollo de Componentes Personalizados:** Diseñar e implementar un mínimo de **dos componentes visuales independientes e instanciables** (por ejemplo, una tabla de datos avanzada con buscador o un widget estadístico).
- **Paso 2.2 - Gestión de Estado y Eventos:** Estos componentes deben ser agnósticos respecto a la vista que los contiene; recibirán los datos a través de propiedades (*Props/Inputs*) y notificarán cambios o interacciones al componente padre mediante la emisión de eventos personalizados (*Callbacks/Outputs*).

FASE 3: Consumo de Datos y Generación de Informes (RA 5)

Integración de la interfaz con los sistemas de persistencia y exposición de métricas.

- **Paso 3.1 - Concurrencia y Asincronía:** Desarrollar servicios que consuman información de manera asíncrona (usando *Promises/Async-Await* o *Observables* de RxJS). Se simulará el consumo dual de fuentes: una API REST basada en bases de datos relacionales (SQL) y un servicio en tiempo real basado en documentos (ej. Firebase Firestore, NoSQL).
- **Paso 3.2 - Dashboard interactivo:** Implementar una vista de "Cuadro de Mando" para el rol Administrador, integrando gráficos visuales (mediante librerías como Chart.js, D3 o equivalentes) que representen datos consolidados.
- **Paso 3.3 - Exportación de datos:** Implementar funcionalidad que permita al usuario generar y descargar informes tabulados del sistema en formatos estándar, concretamente PDF y CSV.

FASE 4: Usabilidad, Accesibilidad y Control de Calidad (RA 4)

Aplicación de las normativas de interacción humano-computadora en interfaces híbridas.

- **Paso 4.1 - Diseño Adaptativo (Responsive):** La interfaz debe ajustarse sin distorsión anatómica a dispositivos móviles, tablets y monitores de escritorio (enfoque *Mobile-First*), utilizando sistemas de grillas y flexbox.
- **Paso 4.2 - Cumplimiento WCAG:** Incorporar contraste de color reglamentario, tamaño de tipografía escalable, control total de la navegación mediante teclado (tabindex lógico) y atributos semánticos WAI-ARIA para garantizar la correcta lectura mediante software de asistencia (lectores de pantalla).

FASE 5: Documentación, Control de Versiones y Despliegue (RA 6 y RA 7)

Preparación del artefacto de software para su entrega a un entorno de producción profesional.

- **Paso 5.1 - Control de Versiones:** El proyecto debe ser inicializado y gestionado íntegramente mediante Git. El alumno debe evidenciar el uso de un flujo de trabajo estructurado (ramas main, develop y ramas de características individuales o feature branches).
- **Paso 5.2 - Documentación Técnica y de Usuario:** Generación automatizada de la documentación del código fuente empleando herramientas estándar (Compodoc para

Angular o JSDoc/TypeDoc para React). Redacción de un Manual de Usuario final integrado en el README o como un componente de "Ayuda" en la app.

- **Paso 5.3 - Despliegue en Producción:** Configurar la integración continua básica y desplegar la aplicación compilada para producción en plataformas cloud como **Vercel**, Netlify o Firebase Hosting.

MATERIALES Y FORMATO DE ENTREGA (ENTREGABLES)

Para que el proyecto sea calificado, el alumno deberá subir a la plataforma de aprendizaje (LMS) del centro un único archivo comprimido .zip con la siguiente nomenclatura: APELLIDOS_NOMBRE_DI_PROYECTO.zip.

Este archivo debe contener exclusivamente un documento PDF con los siguientes enlaces y evidencias:

1. **URL del Repositorio de Código (GitHub/GitLab):** El repositorio debe ser accesible (público o con invitación al docente). El historial de *commits* y el uso de ramas será revisado exhaustivamente para validar la autoría y la evolución del código (RA 7).
2. **URL de la Aplicación en Producción:** Enlace directo al despliegue funcional alojado en Vercel o plataforma equivalente. La aplicación debe ser accesible desde cualquier navegador sin requerir configuraciones locales (RA 7).
3. **Credenciales de Acceso:** Usuarios y contraseñas de prueba debidamente especificados para acceder como Administrador y como Usuario Estándar.
4. **Informe de Accesibilidad y Rendimiento:** Captura de pantalla o exportación en PDF de una auditoría realizada con **Google Lighthouse** evidenciando una puntuación igual o superior a 90/100 en los apartados de *Accessibility* y *Best Practices* (RA 4).
5. **Documentación Técnica Generada:** Carpeta o enlace a la documentación generada automáticamente (TypeDoc/Compodoc), junto con el Manual de Usuario en formato PDF o Markdown (RA 6).

Nota de Integridad: Se evaluará de forma estricta la gestión de la información. Si el alumno utiliza datos sintéticos para las pruebas (recomendado), debe asegurar que en ningún caso se expongan datos personales reales de terceros, aplicando técnicas de anonimización si fuese pertinente. La falta de despliegue o la ausencia del repositorio supondrá automáticamente una calificación de **0 (No Apto)** en la actividad.

Rúbrica de Corrección Técnica

RA y CE Evaluados	Inadecuado (0 - 4.9)	Suficiente / Bien (5.0 - 6.9)	Notable (7.0 - 8.9)	Sobresaliente (9.0 - 10)	Peso
RA 1: Confección de interfaces de usuario. *(CE: b, c, d, e, h)* Arquitectura, enrutamiento y control de acceso.	La arquitectura no sigue un patrón MVVM/MVC. No hay enrutamiento dinámico ni control de acceso (*Guards* / *Private Routes*). El estado global muta de forma incontrolada.	Implementa enrutamiento básico. Diferencia vistas pero la gestión del estado es débil. El acceso protegido es fácilmente eludible desde el cliente.	Arquitectura sólida. Uso correcto de *Lazy Loading*. Implementa interceptores HTTP y protege rutas según el rol (JWT o similar) de forma efectiva.	Implementación impecable del patrón arquitectónico. Gestión de estado predecible (Redux, Context API o NgRx). Seguridad robusta en el enrutamiento y manejo asíncrono de tokens.	**20%**
RA 2: Generación de interfaces declarativas *(CE: a, b, c, d)* Uso de JSX/Templates, *Data Binding* y validaciones.	Código espagueti mezclando lógica de negocio con manipulación directa del DOM. Formularios sin validación reactiva.	Uso aceptable de plantillas (JSX/HTML). Implementa *data binding* unidireccional. Formularios con validación síncrona básica.	Separación estricta de lógica y vista. *Two-way data binding* controlado. Formularios reactivos con validaciones asíncronas correctas (ej. contra API).	Renderizado altamente optimizado (evita *re-renders* innecesarios mediante `useMemo`, `ChangeDetectionStrategy.OnPush`). Formularios reactivos complejos con gestión de errores UX avanzada.	**15%**

RA 3: Creación componentes visuales. *(CE: a, b, d, e, f)* Encapsulamiento, *Props*, *Outputs* y ciclo de vida.	Todo el código se encuentra en vistas monolíticas. No se han desarrollado componentes reutilizables.	Crea al menos un componente reutilizable básico (ej. un botón o input personalizado) que recibe *props*.	Desarrolla componentes complejos independientes (ej. Data Table). Maneja correctamente la emisión de eventos (*Outputs* / *Callbacks*) al componente padre.	Arquitectura basada íntegramente en componentes agnósticos y reutilizables. Uso magistral de los *hooks* o ciclos de vida (useEffect, ngOnInit, ngOnDestroy para evitar *memory leaks*).	**15%**
RA 4: Usabilidad y Accesibilidad. *(CE: a, b, c, d, e)* Responsive Design, WCAG, WAI-ARIA.	Interfaz estática no adaptativa. Fallos graves de usabilidad. Auditoría Lighthouse en Accesibilidad < 50.	Interfaz *Responsive* (Flexbox/Grid). Usabilidad estándar pero carece de atributos semánticos ARIA o navegación por teclado.	Diseño *Mobile-First* fluido. Implementa etiquetas semánticas y alto contraste. Auditoría Lighthouse en Accesibilidad > 80.	Cumplimiento estricto normativas WCAG. Navegación integral por tabulación. Interfaz intuitiva y profesional. Auditoría Lighthouse Accesibilidad y Mejores Prácticas > 95.	**15%**

RA 5: Creación de informes. *(CE: a, b, c, d, e)* Dashboards, consumo de datos y exportación PDF/CSV.	No se integran gráficos ni se consolida la información. El consumo de datos bloquea el hilo principal.	Presenta datos en tablas simples. El consumo asíncrono (SQL/NoSQL mockeado) funciona pero sin manejo de errores. Exportación CSV básica.	Integra un Dashboard visual (Chart.js/D3). Consume datos asíncronos concurrentes con *Spinners* de carga. Exportación a PDF y CSV funcional.	Dashboard interactivo (filtros en tiempo real). Orquestación asíncrona avanzada (`Promise.all` o *RxJS forkJoin*). Exportación robusta de informes tabulados con diseño en el PDF.	**15%**
RA 6: Doc de aplicaciones *(CE: a, b, c, d)* TypeDoc / JSDoc, código limpio y Manual de Usuario.	Código sin comentar. Ausencia de manual de usuario o archivo README descriptivo.	Código comentado superficialmente. README con instrucciones básicas de instalación.	Documentación técnica generada automáticamente (TypeDoc / Compodoc). El manual de usuario detalla el funcionamiento de la app.	Documentación técnica exhaustiva e integrada en el *pipeline* de CI. Manual de usuario embebido en la propia aplicación como sistema de "Ayuda" contextual o Markdown estructurado profesional.	**10%**

RA 7: Distribución y despliegue. *(CE: a, b, d, e, f)* Git Flow, empaquetado, integración en Vercel/Netlify.	Sin control de versiones (o un único *commit*). El proyecto solo corre en `localhost`. Entrega inválida para entorno profesional.	Uso de Git básico. Despliegue manual funcional en Vercel/Netlify, pero sin un flujo de ramas estructurado.	Uso correcto de ramas (`main`, `develop`, `features`). Despliegue automatizado conectado a GitHub. Aplicación compilada para producción (PWA).	Implementación estricta de *Git Flow* con *Pull Requests*. *Pipeline* de CI/CD que bloquea despliegues si fallan los tests unitarios/E2E configurados. Artefacto de producción altamente optimizado.	**10%**

Penalizaciones Automáticas (Criterios de Exclusión):

- **Fallo de Compilación/Despliegue:** Si la URL de producción (RA 7) devuelve un error 500, pantalla en blanco o no resuelve, el proyecto se califica como **0**.
- **Ausencia de Repositorio:** La no entrega del código fuente con su historial de control de versiones anula la evaluación completa.
- **Violación de Datos (Simulados):** Exponer tokens reales de APIs de terceros o contraseñas en texto plano en el repositorio público restará **3 puntos** sobre la nota final.